

Project Report

BoulderBuddy

Team:

Name	Student ID
Deniz Rahnefeld	409637
Peer Schulze	410246

1. Use Case

- Problem statement:

Kletterhallen in Deutschland verwenden meist nicht die großen, bekannten Gradsysteme wie das französische System oder das V-System. Stattdessen kommen fast immer einfache Nummerierungen oder klassische Farben zum Einsatz, um Routen voneinander zu unterscheiden. Erschwerend kommt hinzu, dass eine „8er“-Route in der einen Boulderhalle nicht denselben Schwierigkeitsgrad hat wie eine „8er“-Route in einer anderen — die Skalen sind also nicht einmal untereinander vergleichbar.

Genau hier liegt das größte Problem gängiger Boulder-Apps: Sie bieten keine Möglichkeit, eigene Gradsysteme zu nutzen oder zu erstellen, sondern zwingen dem Nutzer eine universelle Skala auf, die für jede Halle gleichermaßen gelten soll. Darüber hinaus fehlt es an Funktionen, um alle Routen einer Trainings-Session zuzuordnen, sie mit Fotos oder Videos anzureichern und Versuche sowie Notizen dauerhaft zu speichern. Auch ein Hangboard-Timer ist selten integriert — meist gar nicht vorhanden —, sodass die Hangzeit umständlich über eine externe Standard-Timer-App gestoppt werden muss.

All diese Punkte sollen mit BoulderBuddy umgesetzt werden, damit eine Boulder-App endlich an den Sport angepasst ist und Funktionen besitzt, die Nutzer auch tatsächlich benötigen.

- Motivation:

Einer von uns hat vor einigen Monaten mit dem Bouldern angefangen. Der Sport hat ihm von Beginn an großen Spaß gemacht, und so wurde schnell der Wunsch geweckt, den eigenen Fortschritt festhalten zu können. Beim Ausprobieren verschiedenster Apps fiel jedoch auf, dass wesentliche Dinge — allen voran das Erstellen eigener, hallenspezifischer Gradsysteme — schlicht nicht möglich waren. Das ständige Umrechnen und Raten, welcher Schwierigkeitsgrad denn nun zum jeweiligen Boulder passt ist so ein relevanter Bereich, dass wir uns entschieden haben das Projekt BoulderBuddy anzugehen.

- **Real-world relevance:**

Bouldern ist in den letzten Jahren von der Nische zum Breitensport geworden: Seit der olympischen Premiere des Sportkletterns (Tokio 2020) sind vielerorts neue kommerzielle Boulderhallen entstanden, und die Zahl der Aktiven wächst stetig. Damit ist das beschriebene Problem kein theoretisches, sondern begegnet Kletternden in praktisch jeder Halle.

Dass jede Halle ihr eigenes Bewertungssystem pflegt, lässt sich an realen Beispielen belegen: Die Boulderwelt-Hallen (u. a. München, Frankfurt, Regensburg) nutzen eine reine Farbleiter von Weiß über Gelb, Orange, Rot, Blau und Lila bis Schwarz; die DAV-Kletterhallen setzen je nach Standort auf eigene Farb- oder Nummernsysteme. Ein einheitliches, hallenübergreifendes Schema existiert schlicht weg nicht — und wird es realistisch auch nie geben.

Für die kletternde Person bedeutet das im Alltag: Der eigene Fortschritt lässt sich zwischen zwei Hallen kaum vergleichen, und eine App mit fester Universalskala zeigt bestenfalls ungenaue, schlimmstenfalls irreführende Werte an. Eine Anwendung, die frei definierbare Gradsysteme pro Halle erlaubt und das Training vollständig dokumentiert, trifft damit einen konkreten, tagtäglichen Bedarf einer wachsenden Zielgruppe — vom Gelegenheitskletterer bis zum ambitionierten Projektierer.

2. UX Design and Target Group

2.1. Target Group

- **Primary users:**

Personen, die das Bouldern als echtes Hobby und nicht als einmalige Aktion betreiben. Sie gehen zwei- bis dreimal pro Woche in die Halle und wollen sich stetig verbessern: neue Routen ausprobieren und schwierigere davon als Projekte für die nächsten Wochen angehen. Ihr Ziel ist es, leichtere Routen mit immer weniger Versuchen zu schaffen und den Schwierigkeitsgrad ihrer bislang schwersten Route regelmäßig zu steigern. Dabei klettern sie nicht nur in einer einzigen Halle, sondern nutzen verschiedene Hallen mit jeweils unterschiedlichen Gradsystemen.

Für diese Gruppe zählt jeder einzelne Versuch: Sie wollen ihren Fortschritt über Sessions und über Hallen hinweg lückenlos nachvollziehen können — genau das, woran Standard-Apps mit fester Universalskala scheitern.

- **Secondary users:**

Personen, die in unregelmäßigen Abständen bouldern gehen und dabei die Projekte der letzten Session weiter angehen wollen. Sie sind nicht jede Woche in der Halle, aber rund zweimal im Monat anzutreffen. Da zwischen ihren Besuchen viel Zeit vergeht, wissen sie ihre letzten Erfolge und offenen Projekte meist nicht mehr genau — sie nutzen die App vor allem, um nachzuschauen, was ihr letzter Stand war. Auch sie versuchen sich immer wieder an schwierigeren Routen und wollen die leichteren möglichst schnell schaffen.

Für diese Gruppe ist die App weniger ein permanenter Trainingsbegleiter als vielmehr ein Gedächtnis: Sie steigen seltener ein, brauchen dann aber sofort den Überblick über Status, Versuche und Notizen ihrer bisherigen Boulder.

- **User needs:**

Die Nutzer wollen ihren Kletterfortschritt verlässlich festhalten und nachvollziehen — und zwar im Gradsystem der jeweiligen Halle, nicht in einer aufgezwungenen Universalskala. Heute behelfen sie sich, indem sie Grade im Kopf umrechnen und raten, ihre Versuche und Erfolge aus dem Gedächtnis rekonstruieren oder Fotos und Notizen unverbunden über Galerie und Notiz-App verstreuen. Offene Projekte gehen zwischen zwei Hallenbesuchen so regelmäßig verloren. Konkret brauchen sie: (1) frei definierbare, hallenspezifische Gradsysteme, (2) eine sessionbasierte Dokumentation jeder Route mit Foto, Versuchen, Status und Notiz, (3) eine komplette Fortschrittsübersicht über Sessions und Hallen hinweg sowie (4) einen integrierten Hangboard-Timer, der die heutige Behelfslösung mit einer separaten Timer-App ersetzt.

- **Key usage scenario / user story:**

Primary User — Max 23, Student und Hobbyboulderer:

Max klettert zwei- bis dreimal pro Woche und wechselt zwischen zwei Hallen mit unterschiedlichen Farbsystemen. Beim Betreten der Halle startet er in BoulderBuddy eine neue Session und wählt das Gradsystem der jeweiligen Halle. Zu jedem Boulder, den er angeht, legt er einen Eintrag mit Foto, Grad, Versuchen und Status an; eine besonders schwere Route markiert er als Projekt. Nach der Session öffnet er die Statistik, um zu sehen, ob seine Flash-Rate steigt und ob sein schwerster geschaffter Grad sich über die Wochen erhöht. Er öffnet die App also, um während des Trainings zu dokumentieren und seinen Fortschritt hallenübergreifend zu verfolgen.

Secondary User — Klaus 30, Vollzeitangestellter:

Klaus geht nur ein- bis dreimal im Monat mit einem Kumpel bouldern. Weil zwischen den Besuchen viel Zeit vergeht, weiß er beim Ankommen in der Halle nicht mehr, an welchen Routen er zuletzt gescheitert ist. Noch beim Aufwärmen öffnet er BoulderBuddy und schaut in seine letzte Session: Welche Boulder standen auf „Projekt“, wie viele Versuche hatte er schon gebraucht, was hatte er sich notiert? Mit diesem Überblick geht er gezielt seine offenen Projekte erneut an. Er öffnet die App also vor allem, um sich an seinen letzten Stand zu erinnern und dort weiterzumachen.

2.2. Mockups

- **Workflows** (Aufgrund der besseren Lesbarkeit außerhalb der Tabelle):

A — Session Dokumentieren:

Session starten/erstellen (Name, Gradsystem wählen) → Am Hangboard aufwärmen (selbst gewählte Satz und Pausenzeiten) → Boulder Hinzufügen (Fotos/Videos einfügen, Schwierigkeit wählen, Versuche eintragen und Geschafft oder Projekt wählen) → Review über die Statistiken der Session (Anzahl geschaffter Boulder, Versuche gesamt und Top Grade) → Session Beenden

B — Statistiken anschauen und bewerten:

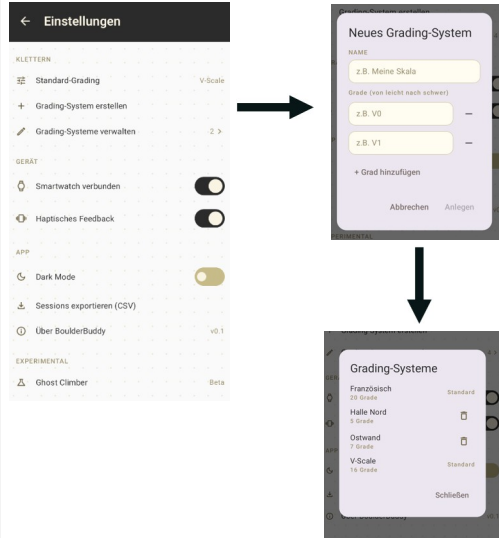
Statistiken → Flash Rate anschauen (Verbesserung zur letzten Auswertung?) → Schwierigkeitsverteilung der Gradsysteme bewerten (Schwierigkeit über die Zeit nach oben verschoben?) → Hangboard-Trainings Statistiken (ggf. alle Workouts anschauen) → Aktivitäten der letzten 4 Wochen (War ich auch so oft wie ich gehen wollte)

C — Gradingsystem erstellen und verwalten:

Einstellungen → Grading-System erstellen (Name und Skala hinzufügen) → Anlegen → Grading-Systeme verwalten → ggf. löschen einer Skala die nicht mehr genutzt wird (bspw. Kletterhalle im Urlaub)

Zum erstellen von Wireframes haben wir Balsamiq genutzt. Darauf basierend hat Claude uns die richtigen Konzeptscreens erstellt. Ein Paar der aufgeführten Mockups wurden auch direkt aus der Vorschau aus AndroidStudio übernommen, da diese relevante Detailunterschiede aufweisen, welche wir in der Entwicklung geändert haben.

Screen	Mockup	Addresses which user need?
Workflow A		Dieser Workflow adressiert aufgrund der Session basierten Dokumentation und der Nutzung des integrierten Hangboard-Timers die User needs 2 und 4.
Workflow B		Dieser Workflow adressiert aufgrund der Analyse der Fortschritte und das nutzen der kompletten Statistikübersicht den User need 3.

Screen	Mockup	Addresses which user need?
Workflow C		Dieser Workflow adressiert aufgrund des erstellen und verwalten von eigenen definierbaren Gradsystemen den User need 1.

2.3. Design Rationale

- What did you deliberately leave out / simplify, and why?

Wir haben uns bewusst gegen Accounts und Registrierung entschieden. Ein solcher Schritt würde die Nutzer unnötig Zeit kosten, ohne einen Mehrwert zu bieten – schließlich werden keine sensiblen oder schützenswerten Daten erfasst, die vor Dritten verborgen bleiben müssten. So kann der Nutzer die App öffnen und ohne Umwege direkt mit seinem Training beginnen.

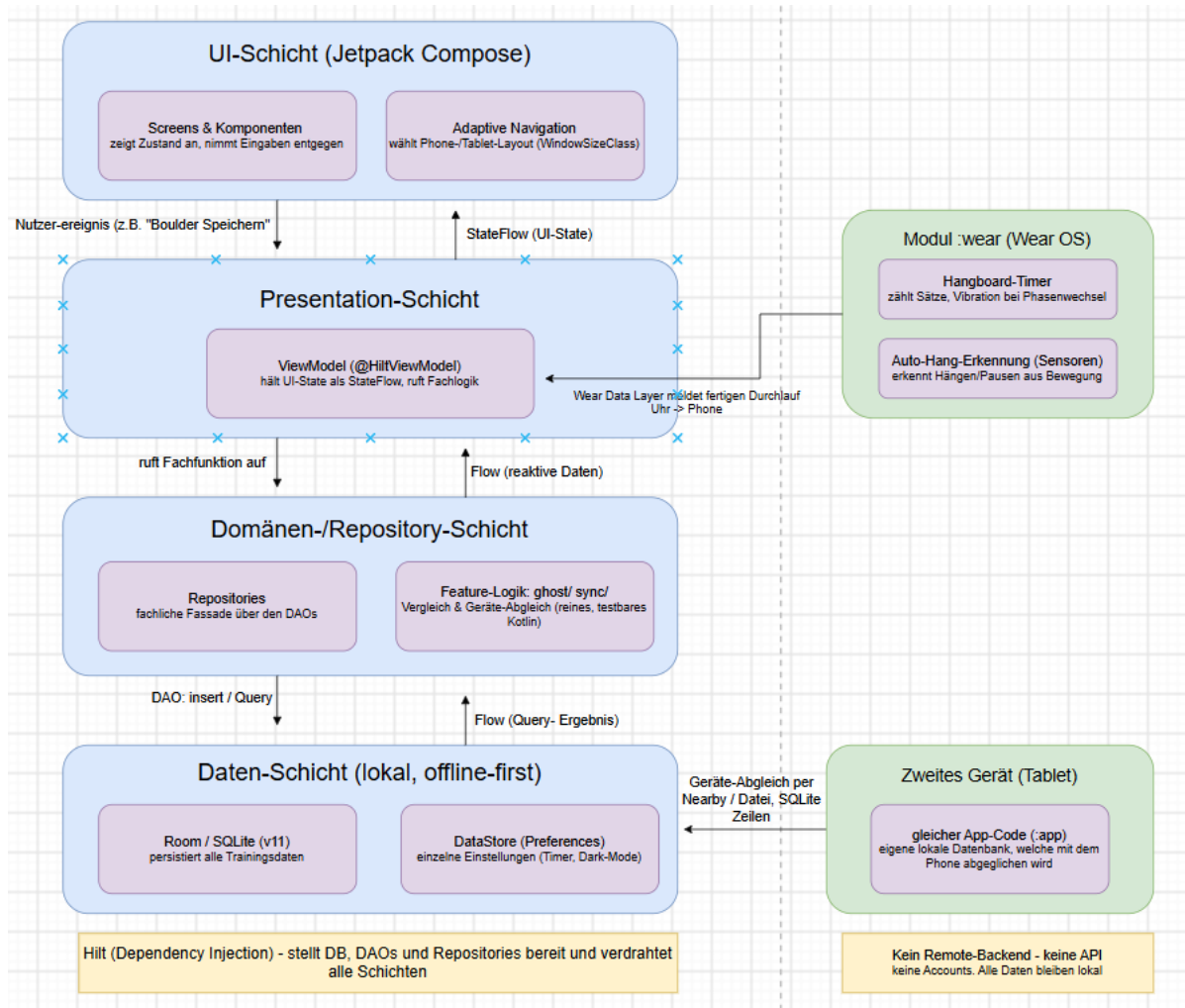
Ebenso bewusst gewählt ist die Rolle der Routenfarben: Die Farbe, die ein Nutzer beim Anlegen einer Route vergibt, kennzeichnet nicht den Schwierigkeitsgrad, sondern dient allein dem leichteren Wiederauffinden der Route an der Boulderwand.

Den Ghost Climber haben wir gezielt in die Einstellungen ausgelagert. Es handelt sich um ein experimentelles Feature, das (noch) nicht vollständig zuverlässig arbeitet. Hinzu kommt, dass es nur für einen kleinen Teil der Nutzer bei jeder Session relevant ist: Interessant wird der Ghost Climber vor allem dann, wenn einem die Ideen ausgehen, warum eine Route nicht gelingt – und damit eher punktuell als dauerhaft genutzt wird.
- What's one UX decision you made that an AI assistant suggested differently?

Eine bewusst gegen den Vorschlag der KI getroffene Entscheidung betrifft den bereits erwähnten Ghost Climber. Anfangs sind wir dem Vorschlag gefolgt und haben das Feature prominent in der Navigationsleiste platziert. Im Laufe der Entwicklung zeigte sich jedoch, dass der Ghost Climber (noch) nicht vollständig zuverlässig arbeitet und eher experimentellen Charakter hat. Hinzu kommt, dass ihn vor allem Nutzer, die nicht auf Leistung oder Wettkampf hin bouldern, nur selten verwenden werden. Da unsere Zielgruppe von Leistungssportlern über Hobby- bis hin zu Gelegenheitsboulderern reicht, wäre ein derart präsender Platz in der Navigationsleiste unpassend. Wir haben das Feature daher bewusst zurückgestuft: Es bleibt jederzeit erreichbar, drängt sich aber niemandem auf, der es nicht braucht.

3. System Architecture Diagram

Die BoulderBuddy Architektur haben wir mit Draw.io erstellt:



3.1. Architecture Justification

- **Why this structure?**

Das Diagramm zeigt vier klar getrennte Schichten (UI → Presentation → Domänen-/Repository → Daten) im für Android empfohlenen MVVM-Muster mit Repository und unidirektionalem Datenfluss: Die zustandslose UI-Schicht (Compose) schickt Nutzer-Ereignisse nach unten, während der Zustand als StateFlow wieder nach oben fließt. Ganz unten ist die App offline-first ohne Backend gebaut (im Diagramm der Kasten „Kein Remote-Backend“) — Bouldern findet in Hallen mit schlechtem Empfang statt, und es fallen keine schätzenswerten Daten an, weshalb Room die Single Source of Truth ist, aus der jede Ansicht reaktiv liest. Einen eigenen domain-Layer haben wir bewusst weggelassen: Die Repositories sind dünne Fassaden über den DAOs, und wo echte Algorithmik anfällt, sitzt sie testbar in der android-freien Feature-Logik (ghost/, sync/). Quer über allem verdrahtet Hilt die Schichten, und die beiden rechten Blöcke zeigen, dass Phone/Tablet sich ein Modul (:app) teilen, während die Uhr ein eigenständiges :wear-Modul ist.

- **What alternative did you consider, and why did you reject it?**

Für „dieselben Daten auf mehreren Geräten" haben wir ein Cloud-Backend mit Konten erwogen, es aber verworfen, weil es dem sofortigen, offline-fähigen Start widerspricht — stattdessen läuft der im Diagramm gezeigte Pfeil „Geräte-Abgleich" direkt zwischen den Geräten über Nearby Connections, wo beide Stände gleichzeitig vorliegen und die App die Richtung selbst bestimmt. Eine automatische Konfliktlösung per Zeitstempel haben wir abgelehnt, weil die Gerätezeit nicht verlässlich ist und so stillschweigend Trainingsdaten überschrieben würden; deshalb entscheidet der Nutzer einmal pro Abgleich über die echten Konflikte. Und in der Feature-Logik des Ghost Climbers haben wir OpenCV zugunsten von schlankem Eigencode (Homographie, DTW) verworfen, um eine große native Abhängigkeit in der APK zu vermeiden.

- **Walk-through**

Aktion aus 2.1 (Workflow A): Max dokumentiert in der laufenden Session einen Boulder mit Foto, Grad, Versuchen und Status.

1. **UI-Schicht** → „**Screens & Komponenten**": Max füllt im Screen die Felder aus (Foto über PhotoPicker/CameraX, Grad, Versuche, Status). Die Adaptive Navigation daneben hat vorher — je nach WindowSizeClass — das Phone- oder Tablet-Layout gewählt.
2. **Pfeil „Nutzer-Ereignis (Boulder speichern)"** → **Presentation-Schicht**: Der Screen meldet das Speichern an das ViewModel (@HiltViewModel), das die Eingaben zu einer RouteEntity bündelt.
3. **Pfeil „ruft Fachfunktion auf"** → **Domänen-/Repository-Schicht**: Das ViewModel ruft das Repository (RouteRepository); Hilt hat die Implementierung injiziert.
4. **Pfeil „DAO: insert / Query"** → **Daten-Schicht**: Das Repository schreibt über das DAO die Zeile in Room / SQLite (v11); die Bilddatei wird nicht kopiert, nur als URI referenziert. (DataStore bleibt hier unberührt — es hält nur Einstellungen.)
5. **Rückweg — Pfeil „Flow (Query-Ergebnis)"** → „**Flow (reaktive Daten)"** → „**StateFlow (UI-State)"**": Room stößt über den beobachteten Flow das Repository und das ViewModel an, das seinen StateFlow aktualisiert; die UI-Schicht rekonstruiert und zeigt den neuen Boulder sofort — auf dem Tablet im zweiseitigen List-Detail, auf dem Phone einspaltig.
6. **:wear (optional parallel)**: Läuft nebenbei ein Hangboard-Durchlauf, kommt er über den Pfeil „Wear Data Layer — meldet fertigen Durchlauf Uhr→Phone" in die Presentation-/Datenschicht und wird an dieselbe Session gehängt.
7. **Zweites Gerät (Tablet)**: Beim späteren „Geräte-Abgleich per Nearby / Datei" wandert der neue Boulder als SQLite-Zeile auf das Tablet — ganz ohne Backend.

3.2. Third-Party Libraries

Library	Purpose	Why this one (over alternatives)?
Jetpack Compose (BOM)	Deklaratives UI-Toolkit für Phone/Tablet (steuert ui, ui-graphics, Tooling, Test)	Aktueller Android-Standard, ein Framework für alle Formfaktoren. Alt: XML-Views + ViewBinding: mehr Boilerplate, kein gemeinsamer Stil mit Wear
Material 3	Design-System, semantische Farb-/Typo-Tokens, M3-Komponenten	Offizielles Material-Design für Compose, Light/Dark über Tokens

Library	Purpose	Why this one (over alternatives)?
Material3 Adaptive (+ window-size-class)	Tablet-Layouts: ListDetailPaneScaffold, WindowSizeClass-Breakpoints	Offizielle adaptive Bausteine; Eigenbau der List-Detail-Logik wäre fehleranfälliger. Eigene Versionslinie (nicht im BOM)
Compose for Wear OS (compose-material / -foundation / -navigation)	UI der Smartwatch	Wear-optimierte, eigene Material-Bibliothek; Phone-material3 passt nicht auf runde, kleine Displays
androidx.wear	Wear-OS-Basis-Support	Grundbaustein des Wear-Moduls
Navigation Compose	Type-safe Navigation (Routen als @Serializable-Typen)	Compile-sichere Argumentübergabe statt String-Routen
kotlinx.serialization.js on	Serialisierung der Navigations-Routen + Ghost-Artefakte	Kotlin-nativer, reflexionsfreier Serializer
Coil 3 (coil-compose, coil-network-okhttp)	Bild-Laden/-Anzeige in Compose (AsyncImage)	Leichtgewichtig, Compose-nativ. Alt: Glide/Picasso: View-orientiert, schwerer
Glance App-Widget (appwidget, material3)	Homescreen-Widget mit laufender Session	Einzige Compose-artige Widget-API; eigene Versionslinie
Room (runtime, ktx, KSP-Compiler)	Lokale SQLite-DB (v11), reaktive Flow-Queries, Schema-Export, echte Migrationen	Offizielle, stabile Persistenz mit Compile-Zeit-geprüftem SQL. Alt: rohes SQLite: keine Typ-/Query-Sicherheit
DataStore (Preferences)	Einzel-Einstellungen: Timer-Config, Gradsystem, Dark-Mode, Geräte-ID	Für wenige Schlüssel ist eine DB-Tabelle überzogen; moderner SharedPreferences-Ersatz (async, Flow)
Hilt (android, compiler)	Dependency Injection über alle Schichten + Android-Services	Offiziell empfohlen, wenig Boilerplate über Dagger.
Hilt Navigation Compose	@HiltViewModel in Compose-Screens beziehen	Standard-Brücke Hilt ↔ Navigation Compose
CameraX (core, camera2, lifecycle, video, view)	Eigener Aufnahme-Screen für Foto/Video (Boulder & Ghost), feste Auflösung (Quality.HD)	Berechenbarer als der geräteabhängige Kamera-Intent; legt app-eigene FileProvider-URLs ab. Eigene Versionslinie
Media3 / ExoPlayer (exoplayer, ui)	Video-Wiedergabe im Boulder-Detail & Ghost Climber (lifecycle-aware)	Googles Nachfolger des MediaPlayer; robuste PlayerView via AndroidView
MediaPipe Tasks Vision	Pose-/Skelett-Erkennung im Ghost Climber (PoseLandmarker,	Echtes visibility/presence pro Landmark + internes Tracking (Basis der Filterkette); ersetzte

Library	Purpose	Why this one (over alternatives)?
	RunningMode.VIDEO, 33 Landmarks)	ML Kit. Kein OpenCV — Homographie/DTW sind Eigencode
play-services-wearable	Wear Data Layer: MessageClient (Uhr→Phone) + DataClient (Phone→Uhr)	Standard-Kanal zwischen gekoppelten Geräten; bewusst minimal genutzt
play-services-nearby	Phone↔-Tablet-Abgleich per Nearby Connections (P2P)	Android-nativer Direkt-Transport ohne Konto/Server; beide Stände liegen gleichzeitig vor
play-services-location	Gym-Näherungs-Push: FusedLocationProviderClient + Geofencing	Beide APIs aus einem Artefakt; Standard für energiesparende Standort-/Geofence-Erkennung
error_prone_annotations	Vom Hilt/Dagger-generierten ServiceComponent-Code referenziert (@CanIgnoreReturnValue)	Unter AGP 9 nur transitiv compileOnly da → für den generierten Service-Code explizit nötig, sonst bricht der Build
Material Icons (core, extended)	Icon-Sätze für die Compose-UI	Offizielle Material-Symbole, kein eigenes Icon-Set nötig
AndroidX Core-KTX	Kotlin-Erweiterungen fürs Android-Framework	Standard-Basis jeder modernen Android-App
AndroidX Activity Compose	Compose-Einstieg (setContent), Ergebnis-APIs (PhotoPicker, Permissions)	Offizielle Compose-Activity-Integration
AndroidX Lifecycle (runtime-ktx, runtime-compose, viewmodel-compose)	ViewModel-Anbindung, lifecycle-bewusstes collectAsStateWithLifecycle	Grundlage des MVVM-/StateFlow-Musters
JUnit 4	Test-Runner (JVM + instrumentiert)	De-facto-Standard im Android-Toolchain
kotlinx-coroutines-test	Virtuelle Zeit für ViewModel-/Timer-Tests (advanceTimeBy, runCurrent)	Deterministische Coroutine-Tests ohne echte Wartezeit
Truth	Fluent Wert-Assertions	Klarere Fehlermeldungen als plain JUnit
Turbine	Assertions auf Flow-Emissionen	Lesbarer als manuelles Flow-Sammeln
androidx.arch.core:core-testing	Synchrone Ausführung von Architektur-Komponenten im Test	Standard für Arch-/LiveData-Tests
room-testing	DAO-/Migrations-Tests gegen	Testet echtes SQLite-/FK-

Library	Purpose	Why this one (over alternatives)?
	echte In-Memory-Room-DB	Verhalten ehrlicher als ein Mock; MigrationTestHelper prüft Migrationen
AndroidX Test ext-junit	JUnit-Runner für instrumentierte Tests	Standard-Harness für Gerätetests
Espresso Core	Instrumentierte UI-Interaktionen	Android-Standard für UI-Tests (ergänzend zu Compose-Test)
compose-ui-test-junit4 (+ ui-test-manifest)	Render-/Interaktions-Tests der Compose-UI	Offizielles Compose-Test-Harness

4. Developer Diary

4.1. Development Milestones

#	Date	Activity	Result	Issues
1.0	2026-05-01	Projekt aufgesetzt: App-Idee und Vision festgehalten, Anforderungen und User Stories geschrieben, Technik-Entscheidung getroffen (Jetpack Compose, Room-Datenbank, Hilt, Wear OS)	Konzept + Anforderungen stehen, Stack festgelegt	Keine
1.1	2026-05-07	Erste Wireframes (Bildschirm-Skizzen) in Balsamiq entworfen — grob per KI aus dem Vault vorgeneriert, dann von Hand angepasst; jeder einen davon in Compose nachgebaut	Wireframe-Grundlage + erste vergleich Screens	Einheitliches Gestalten bei aufgeteilter Arbeit schwieriger als angedacht
1.2	2026-05-12	Unsere getrennten Umsetzungsversuche verglichen, Design abgestimmt, App-Logo mit Gemini erstellt	Konkretes Design abstimmen	Screens uneinheitlich bei geteilter Arbeit → Konzept nötig
2.0	2026-06-18	Android-Studio-Projekt neu angelegt und die wiederverwendbaren Grundbausteine gebaut	Projektgerüst und Kernkomponenten	Kleinere Design-abweichungen von den Konzeptscreens
2.1	2026-06-21	Restliche Bausteine	Fertige Komponentenliste,	Grundverständnis

#	Date	Activity	Result	Issues
		ergänzt und die ersten beiden Bildschirme zusammen gebaut	Home Screen	bekommen wie man die Komponenten zusammenführt
2.2	2026-06-24	UI Screens erstellen abgeschlossen	Fehlende UI Screens fertiggestellt → MVP-UI komplett	Kleinere Designfragen die zum Konzeptscreen verändert werden mussten (Button Platzierung, Farben)
2.3	2026-07-03	Bestandsaufnahme gemacht und einen Umbauplan geschrieben (Reihenfolge: erst Navigation, dann Daten). Danach alle Bildschirme miteinander verbunden — vorher startete die App leer, die Screens waren nur einzeln vorhanden	Erstmals durchklickbarer Prototyp; Plan-Datei als gemeinsame Fortschritts-Liste	Android-Gradle-Plugin 9 verurteilte die geplanten Bibliotheks-Versionen nicht → Hilt musste angehoben werden
2.4	2026-07-03	Datenbank (Room) mit Tabellen, Zugriffs-Schicht und Beispieldaten angelegt, Abhängigkeits-Verwaltung (Hilt) eingerichtet und alle Bildschirme von Platzhaltern auf echte Daten umgestellt	Lauffähiges MVP auf dem Handy — Sessions und Boulder überleben den Neustart	Rückkehr auf den Start-Tab landete auf dem falschen Bildschirm (Falle in der Navigations-Bibliothek); noch keine automatischen Tests
2.5	2026-07-04	Beim Durchklicken gefundene Lücken geschlossen: Grad-Systeme (V-Scale, Französisch, eigene) sind jetzt wirklich wählbar und werden an der Session gespeichert, Farbe von der Schwierigkeit entkoppelt, Filter und Statistik arbeiten pro Grad-System	Grad-Systeme durchgängig nutzbar, eigene Systeme anlegbar; Datenbank auf Version 4	Grade verschiedener Systeme sind nicht vergleichbar → der „Top-Grad“ musste überall innerhalb eines Systems berechnet werden
3.0	2026-07-04	Eigene Smartwatch-App (Wear OS) als Begleiter zur Handy-App gebaut: Hangboard-Timer mit Hängen→Pause→Fertig-Ablauf und Vibration, der fertige Durchläufe wird ans Handy gemeldet	Hangboard Timer auf der Smartwatch nutzbar	Hilt zerschoss den Build, sobald der Uhr-Dienst dazukam → mit error_prone_annotations gelöst
3.1	2026-07-04	Automatische Tests eingerichtet — schnelle Tests ohne Emulator für die Logik, echte Geräte-Tests für Datenbank und	8/8 schnelle + 12/12 Geräte-Tests grün	Keine

#	Date	Activity	Result	Issues
		Oberfläche		
3.2	2026-07-04	Routen mit der Funktion ergänzt, dass sie nun auch Videos speichern können.	Routen können Videos speichern und abspielen	Keine
3.3	2026-07-04	Tablet-Layout gebaut: auf großen Bildschirmen stehen Sessions-Liste und Detail nebeneinander, die Statistik wird mehrspaltig	Anforderung „Responsive Layouts: Handy + Tablet“ erfüllt	Sichtprüfung auf einem echten Tablet stand noch aus
3.4	2026-07-04	Timer-Voreinstellungen fürs Hangboard angebunden (die Datenstruktur dafür lag ungenutzt im Projekt) und einen CSV-Export für Sessions gebaut	Eigene Timer-Vorlagen speicherbar, Sessions exportierbar	Keine
3.5	2026-07-04	Drei Komfort-Funktionen ergänzt: Dunkelmodus (folgt dem System, per Schalter umstellbar), Spracheingabe für Notizfelder und ein Startbildschirm-Widget mit der laufenden Session	Dunkelmodus, Diktier-Knopf und Widget vorhanden	Beides später überarbeitet: der Dunkelmodus war am Gerät praktisch unlesbar (erst Anfang August behoben), die Spracheingabe lief über einen fremden Google-Dialog und wurde ersetzt
4.0	2026-07-05	„Ghost Climber“ gebaut — legt zwei Kletter-Videos übereinander, um Versuche zu vergleichen. Skelett-Erkennung per Google-Bibliothek, die eigentliche Vergleichs-Mathematik (Perspektiv-Ausrichtung, Zeit-Synchronisation) selbst programmiert; Ankerpunkte werden von Hand getippt	Ghost Climber Läuft in 4 geführten Schritten, 28 Tests grün	Automatische Griff-Erkennung ohne OpenCV zu unzuverlässig → bewusst nur manuelles Antippen; Technik-Wahl stand unter Prof-Vorbehalt
4.1	2026-07-05	Skelett-Anzeige des Ghost Climbers stabilisiert: von Google ML Kit auf MediaPipe gewechselt und eine Filterkette gebaut, damit das Skelett nicht mehr zittert, blinkt oder Arme/Beine vertauscht	Deutlich ruhigeres Skelett, neue Tests grün	Skelett verschwand bei einzelnen Aussetzern; Qualitätswert der Bibliothek war irreführend; Kletterposen sind für die KI ungewohnt
4.2	2026-07-06	Skelett-Erkennung des Ghost Climbers nachgeschärft: genaueres (dafür langsames)	Ghost Climber in den Hauptzweig übernommen	Der Geist blieb trotzdem unruhig — die eigentliche Ursache (ein Takt-

#	Date	Activity	Result	Issues
		Erkennungs-Modell und robusterer Umgang mit Größen- und Positionssprüngen		Fehler einmal pro Sekunde) wurde erst am 01.08. gefunden
5.0	2026-07-06	Grundsatz-Entscheidung: Hangboard-Training überall gleich behandeln (Handy + Uhr, manuell + automatisch), ein gemeinsames Datenmodell, und jedes Training immer speichern — mit laufender Session verknüpfen wenn möglich, sonst als eigenständiges Training	Verbindliche Architektur Linie	Alte Regel „ohne Session blockieren“ war widersprüchlich und wurde verworfen
5.1	2026-07-06	Neues gemeinsames Datenmodell umgesetzt und eine eigene Verlaufs-Ansicht gebaut, in der auch Hangboard-Trainings ohne Kletter-Session sichtbar sind	Builds + Tests grün, eigenständige Trainings sichtbar	Keine
5.2	2026-07-06	Uhr zeichnet jetzt Bewegungsdaten der Sensoren auf; darauf eine Erkennungs-Logik gebaut, die aus den Rohdaten automatisch Hänge-Sätze erkennt (bewusst ohne Android-Abhängigkeit, damit testbar)	Sensor-Aufzeichnung auf der Uhr (Dienst + Log-Screen + Export) und geprüfte Logik zur automatischen Satz-Erkennung — 6 Unit-Tests grün	Sensoren nur am echten Gerät prüfbar — Emulator liefert keine Bewegungsdaten
5.3	2026-07-06	Live-Bildschirm auf der Uhr gebaut, der Sätze automatisch mitzählt; Uhr und Handy tauschen Ergebnisse und Timer-Vorlagen aus	Live-Erkennung auf der Uhr (Echtzeit-Status, Satz-Zähler, Vibration, Modus-Wahl manuell/auto) plus automatischer Ergebnis- und Vorlagen-Abgleich zwischen Uhr und Handy.	Zusammenspiel Uhr↔Handy + Kalibrierung braucht echte Hardware
6.0	2026-07-07	Push-Erinnerung gebaut, die fragt „Bist du in der Halle? Session starten?“, wenn man länger an einer gespeicherten Kletterhalle ist. Dazu: Hallen mit Koordinaten speichern, Besuchsmuster lernen, kluge Regeln gegen Spam, und ein Hallen-Bearbeiten-Bildschirm mit „Standort übernehmen“-Knopf	Fertige Push Benachrichtigung und 26 Tests grün	Standort-im-Hintergrund-Berechtigungen sind je Android-Version verschieden und heikel
7.0	2026-08-01	Ghost Climber weiter	Stabileres Ghost Climber	Mehrere versteckte

#	Date	Activity	Result	Issues
		verbessert: flüssigere Zeit-Synchronisation und ein „starres Skelett“, das anatomisch unmögliche Knochenlängen korrigiert; dazu neue Messwerte eingeführt, um Qualität überhaupt beurteilen zu können	Feature mit 109 grünen Tests verifiziert.	Fehler entlarvt (Regelkreis der Erkennungs-Box, Rundungsfehler, falsche Reihenfolge, wanderndes Skelett)
7.1	2026-08-01	Ein Ruckeln „einmal pro Sekunde“ aufgespürt und behoben; die Füße ins starre Skelett aufgenommen und einen ehrlichen Qualitäts-Messwert („Puls“) eingeführt	105 Tests grün, stabiler Stand als Git-Tag gesichert	Störung war lange in keinem Messwert sichtbar — Ursache erst durch Auswertung der Rohdaten vom Gerät gefunden
8.0	2026-08-02	Mit einer Garmin-Sportuhr (App „RawLogger“) echte Bewegungsdaten aufgenommen und die bisher selbst erfundenen Testdaten der Hangboard-Erkennung dagegen ersetzt	Echte Test-Aufnahmen, 9 Tests grün; Baustelle bis Wear-Hardware angekommen ist pausiert	Erkennung maß versehentlich die Pausen statt die Sätze (Vorzeichenfehler); erste Aufnahme war leer; Feinjustierung der Schwellen noch offen + Keine Wear Uhr da keine vorhanden hat
9.0	2026-08-03- 2026-08-05	App durchgängig verfeinert und optimiert: funktionslose Bedienelemente entfernt oder mit echter Funktion hinterlegt und die Hangboard-Statistik überarbeitet. Darüber hinaus das Designsystem professionalisiert (Textskala, Light- und Dark-Mode über mehrere Geräte-Runden korrigiert), abgeschnittene Dialog-Ecken behoben, den System-Dialog-Fallback der Spracheingabe aus Datenschutzgründen entfernt sowie zwei Verlaufs-Diagramme	Komplett optimierte App: alle sichtbaren Elemente besitzen eine Funktion, konsistentes und kontrastgeprüftes Farb-/Typo-System in beiden Modi und neue Verlaufs-Diagramme in der Statistik	Fast jeder Fehler wurde erst am Gerät sichtbar, nicht durch Tests: der selbst geschriebene Kontrast-Test blieb grün, während Texte schlecht lesbar waren (Lesbarkeit ≠ Kontrast), der Dark Mode brauchte vier Optimierungsrunden
9.1	2026-08-03	Spracheingabe und Kamera von Grund auf neu gebaut: Diktierfunktion von Googles fertigem Dialog	Eigene Diktier- und Aufnahme-Oberfläche statt Fremd-Dialoge und 162 Unit-Tests grün.	Keine

#	Date	Activity	Result	Issues
		auf die SpeechRecognizer-API mit eigener Aufnahme-UI und zweistufige Fallback-Kette umgestellt. Foto/Video-Aufnahme vom Kamera-Intent auf einen eigenen CameraX-Screen		
9.2	2026-08-05	Tablet-UI nach Android Best Practices neu gebaut: adaptives Zwei-Spalten-Layout mit eigenem SideNav ab 600 dp, ListDetailPaneScaffold, gedeckelte Kachel-, Heatmap- und Balkenbreiten	Professionelles Tablet-Layout	Zwei Layout-Fehler nur per Screenshot gefunden (fillMaxWidth().width In deckelt nicht; Balken fiel auf Breite 0)
9.3	2026-08-07	Geräte-Abgleich zwischen Handy und Tablet gebaut: beide Geräte tauschen ihren kompletten Datenstand direkt per Nearby Connections (P2P, ohne Konto/Server) aus — Verbindung, Übertragung als Datei, Zusammenführen gegen die echte SQLite-Datenbank, mit eigenem Vordergrund-Dienst. Datenbank dafür abgleichbar gemacht (Migrationen, Herkunfts-Kennzeichnung, geräteunabhängige Verweise)	Zwei Geräte gleichen ihren Stand direkt miteinander ab; beide Stände liegen gleichzeitig vor	Sieben Fehler erst im Ablauf-Durchgang gefunden, zwei davon mit Datenverlust — vor dem Merge behoben
10	(TODO: Datum wenn die Dok fertig ist)	Einzelne Abschnitte der Dokumentation durchgehen und mittels der Entwicklungsdokumentation des Obsidian Vaults bearbeiten	Vollständige Dokumentation mit komplett bearbeiteten Abschnitten	Claude hat teilweise nicht alle relevanten Teile zu den Dokumentationsabschnitten aus dem Vault gefiltert, weshalb immer doppelt drüber geschaut werden musste, ob Claude sein Output auch vollständig ist.

4.2. AI Interaction Log — Critical Incidents

#	Milestone	What happened	AI's output (brief)	Your action / validation
1	1.0	Vault Struktur Initialisiert und alle Startnotizen angelegt	Vollständige Ordnerstruktur von Projektübersicht über Design bis hin zur Dokumentation und 14 Startnotizen wie Beispielsweise die App-Idee eingetragen	Struktur und die 14 Startnotizen gesichtet und übernommen. Das von der KI falsch angesetzte Abgabedatum korrigiert (Juli → August)
2	1.0	Zur besseren Transparenz wurde eine KI-Nutzung.md angelegt. Jede KI-Nutzung wird dort direkt eingetragen	Vorliegende Datei mit Regelwerk, Format-Vorlage und rückwirkenden Einträgen	Die KI-Aktionen gesichtet und überprüft. Seither alle Nutzungen eingepflegt
3	2.0	Finalisierte Screen-Konzepte, Navigationsstruktur und Komponenten-Inventar in den Vault eintragen und dabei Farbpalette & Theme fertigstellen	Screen Konzepte, Navigation, Komponenten Liste und Farbpalette erstellt und am Ende in die dazugehörigen Dateien des Vaults eingetragen	Das Design der Screen Konzepte selbst erarbeitet. Die Farbpalette und Komponenten mit Claude diskutiert und die Einträge im Vault überprüft
4	2.1	Alle UI Komponenten die zur Erstellung der Screens nötig sind gebaut. Die Komponenten wurden mittels der zuvor erstellten Liste abgearbeitet	Alle in der Liste stehenden Kotlin-Komponenten (RouteCard, SessionListItem, ...) im gemeinsamen Stil	Alle Komponenten nach Erstellung durchgeschaut und mittels der Preview validiert, dass diese auch nach UI-Konzept gebaut wurden.
5	2.1→2.2	Bauen aller UI Screens mittels der bereits gefertigten Komponenten	Screens teils komplett gebaut, teils eigene angefangene Implementierung vervollständigt. Platzhalter durchgängig mit TODO markiert	Teile/komplette der Screens selber gebaut. Die KI hat Fehler ausgebessert. Alle Screens und Änderungen überprüft und mittels Preview Aussehen validiert
6	2.2	Alle Screens gegen die Konzept-Screens validieren und Designanpassungen umsetzen — u. a. der Dropdown-Umschalter Sessions ↔ Boulder-Übersicht	UI-Konzept-Screens mit umgesetzten Screens verglichen und Fehler wie falsch genutzte Farben oder ungenutzte Importe ausgebessert	Erneut die Preview angeschaut und selber mit den Konzept-Screens des Vaults verglichen

#	Milestone	What happened	AI's output (brief)	Your action / validation
7	2.3	Die Screens waren fertig aber es gab keine Navigation und keine Datenschicht. Die KI sollte dafür eine Bauplan schreiben	Fertiger Implementierungsplan mit gegebener Ausgangslage und gegliederten Phasen von 0-7	Validieren der einzelnen Phasen. Entscheidung das Jede Phase mittels eigenen Branch bearbeitet wird
8	2.4	Die ersten drei Plan-Phasen umsetzen — Bibliotheken einbinden, Navigation bauen, Datenbank (Room) anlegen. Je Phase ein eigener Branch und Pull Request	Version-Katalog, klickbare Navigation und Datenschicht mit Beispieldaten; MainActivity zeigt erstmals echte Inhalte.	Validiert und zwei nötige Abweichungen (Hilt-Version anheben wegen Android-Gradl-Plugin 9) wurden im Plan vermerkt und korrigiert. Jede Phase wurde Getestet
9	2.4	Phasen 4–6: die Screens von Platzhaltern auf echte Datenbank-Daten umstellen (Hilt, Repositories, ViewModels)	Größter Einzelcommit des Projekts (34 Dateien); ab hier überleben Sessions und Boulder den App-Neustart — das Handy-MVP läuft.	Die Navigation und Speicherung der Daten live in der App getestet. Dabei ist aufgefallen, dass die Grad System Auswahl wirkungslos war. Behebung direkt im Anschluss
10	3.0 ff.	Aus dem groben „Phase 7“-Block einen kleinschrittigen Plan für die restlichen Features machen.	Fertiger Implementierungsplan für die Phase 7. Phasen 7.1-7.6: Tablet Layout, Wear OS, Vider Support und Timer Voreinstellung Dark Mode, Ghost Climber und erste automatische Tests	Letzte Rückfragen mit Claude für die Umsetzung und das Design der Features geklärt, um anschließend die Umsetzung zu erleichtern.
11	3.0-3.5	Phase 7 umsetzen. Jede Phase der 6 gegebenen wurde nacheinander ausgeführt und validiert	Die fertigen Features wie Tablet Layout, Watch App, Dark Mode, etc. in Android Studio umgesetzt	Verifiziert waren nur Build und ab Test-Phase die Tests — keine Sichtprüfung am Gerät; die Folgen zeigten sich später (Dark Mode unlesbar, Spracheingabe später ersetzt)
12	4.0	Den Ghost Climber aus einem Prosa-Einfall in ein tragfähiges Konzept überführen	Strukturierte Problemanalyse und Pipeline-Entwurf; alle unsicheren Größen	Die zentrale Klarstellung geliefert — Griffe dienen nur als Ausrichtungen-

#	Milestone	What happened	AI's output (brief)	Your action / validation
			ausdrücklich als offen markiert	Anker, nicht als erkannte Route — und das Feature blieb im Status „Idee“, bis die Freigabe zum Bau gegeben wurde
13	4.0	Den Ghost Climber vom Konzept in eine erste lauffähige Pipeline bringen	fünf lauffähige Bau-Stufen — mit bewusst reduziertem Umfang (eigene Mathematik statt der schweren OpenCV-Bibliothek, zwei frei gewählte Videos), was den drohenden Blocker auflöste	Weil die offenen Fragen vorab geklärt waren, lief das ohne Rückfrage; geprüft war aber nur der Build
14	4.1	Die App stürzte beim Ghost-Icon ab, und das erkannte Skelett zitterte und vertauschte Arme/Beine. Die Erkennung ließ sich am Entwickler-Rechner nicht ausführen	Statt zu raten, hat die KI echte Videobilder ausgewertet und belegt: das Subjekt ist im Bild zu klein → Erkennungsqualität, kein Rechenfehler. Daraus Absturz-Fix und Wechsel der Erkennungs-Bibliothek auf MediaPipe	Ruhigeres Skelett, neue, durchgeführte Tests grün
15	4.2	Skelett-Erkennung weiter nach schärfen und den Ghost Climber in den Hauptzweig (main) übernehmen	Wechsel auf ein genaueres, aber langsames Erkennungsmodell, robuster gegen Größen- und Positionssprünge.	Der Geist blieb trotzdem unruhig — die eigentliche Ursache (ein Takt-Fehler einmal pro Sekunde) wurde erst am 01.08. gefunden, mit den damaligen Messwerten grundsätzlich unsichtbar

#	Milestone	What happened	AI's output (brief)	Your action / validation
16	5.0-5.3	Die automatische Hangboard-Erkennung auf der Smartwatch bauen, samt gemeinsamem Datenmodell für manuelles und automatisches Training	Statt die Uhr nachfragen zu lassen, gilt jetzt „immer speichern, verknüpfen wenn möglich“ (aktive Session → anhängen, sonst eigenständiges Training, nie verworfen). Umgesetzt mit einem gemeinsamen Datenmodell für manuelles und automatisches Hangboard	Schwierige Validierung da keine Testuhr vorhanden war
17	6.0	Push-Benachrichtigung, wenn man sich einer gespeicherten Boulderhalle nähert — erst als Konzept entwickelt, dann gebaut	Komplette Standort-Erkennung (Geofencing), Besuchs-Logik und Benachrichtigung mit direktem Sprung in den vorbefüllten Session-Screen	Ausdrücklich als ungeprüft akzeptiert (ohne echte Ortswechsel nicht testbar)
18	7.0/7.1	Den Ghost Climber am echten Gerät verfeinern — Ausgangspunkt: das Skelett wackelte weiterhin, obwohl alle Messwerte gut aussahen	Die KI holte die Rohdaten vom Gerät und rechnete selbst nach — das enttarnte eine verdeckte Störung genau einmal pro Sekunde, die über jede zusammengefasste Zahl unsichtbar war	Selbst Analysieren am Gerät versucht und die KI bei Berechnungen korrigiert. Zielvorgabe geändert, da diese zu streng gesetzt war
19	9.0	Die Basis App verfeinern: funktionslose Bedienelemente entfernen oder funktional machen, Dark Mode überarbeiten, 0 min Anzeige in den Hangboard Statistiken ändern	Nicht Funktionale Buttons und andere Bedienelemente zu funktionierenden Elementen verändert. Hangboard Statistik so geändert, dass Statistiken auch vernünftig auslesbar sind	Build überprüfen und Unit Tests laufen lassen. Laufend noch nicht getestet
20	9.0	Über mehrere Runden war der automatische Farb-Kontrast-Test grün	Die KI hat die Farben nicht nach Gefühl geändert, sondern	Jede Runde am echten Gerät gegengeprüft, ob der

#	Milestone	What happened	AI's output (brief)	Your action / validation
		— am echten Handy waren viele Texte aber trotzdem schlecht lesbar, besonders im Dark Mode.	nachgerechnet und die echten Ursachen gefunden: ein Farbwert, der zwei Aufgaben gleichzeitig erfüllte, eine vergessene Design-Rolle, eine falsch gewählte Schriftgröße und eine Trennlinie, die im Code stand, aber nie sichtbar war.	Dark Mode gut aussieht.
21	9.2	Beim Umbau auf das Tablet-Layout hat sich die KI zum ersten Mal ihre eigenen Ergebnisse per Screenshot vom Emulator angeschaut, statt sie nur zu berechnen.	Dabei kamen zwei Fehler heraus, die kein Test gefunden hätte: eine Größenbegrenzung, die gar nicht wirkte, und danach eine Korrektur, die die Balken im Diagramm komplett verschwinden ließ — obwohl Build und Tests grün waren.	Wir haben darauf bestanden, dass die KI den Emulator nutzt und ihre Aussagen selbst überprüft. Beide Fehler waren nur im Screenshot zu sehen; danach wurden die Tests gezielt gegen genau diese Fehler geschrieben.
22	9.3	Den Geräte-Abgleich Handy ↔ Tablet planen und bauen: Zwei Geräte sollen ihren kompletten Datenstand direkt austauschen — ohne Konto, ohne Server.	Die KI prüfte den echten Gerätestand und wählte Nearby Connections (P2P). Fünf Plan-Runden deckten drei Fehler auf, die jeden Abgleich zerstört hätten (Ghost-Dauerkonflikt, divergierende Herkunft, alte WAL überschreibt neue DB) → „Gedächtnis statt Ersetzen“, Migrationen, WAL-Checkpoint. Beim Bauen sieben weitere, zwei mit Datenverlust.	Den Plan Runde für Runde mitgeprüft und die Grundregel bestätigt, dass die App die Abgleich-Richtung selbst bestimmt (beide Stände liegen gleichzeitig vor). Die zwei Datenverlust-Fehler vor dem echten Zusammenführen gegen echtes SQLite abgesichert und den Abgleich abschließend zwischen zwei Geräten durchgespielt.
23	10	Raus suchen der relevanten Abschnitte des Obsidian Vaults für die einzelnen Dokumentationsschritte und Korrigieren der Dokumentation	Ausgeben der relevanten Abschnitte und Formulierungshilfen sowie Sprachliche Verbesserungen	Auf Basis der gegebenen Abschnitte die Dokumentation geschrieben und die Sprachlichen Verbesserungen angewendet.

5. Android Features Report

5.1. Features Used

Feature	Where it's used (screen/class)	Why this feature was needed
Room / SQLite (DB V11)	Die zentrale Datenbank hinter allem — Hallen, Sessions, Boulder, Grade, Hangboard-Workouts (BoulderBuddyDatabase, 11 Tabellen)	Speichert alle Trainingsdaten dauerhaft auf dem Gerät. Nach App-Neustart ist alles wieder da, ganz ohne Server.
DataStore	Einstellungen (Settingsrepository): zuletzt genutzte Timer-Werte, gewähltes Gradsystem, Dark-Mode	Für einzelne Einstellungen, für die eine ganze Datenbank-Tabelle übertrieben wäre.
System-Photopicker	„Foto/Video wählen“ beim Anlegen einer Route, in den Einstellungen und im Ghost Climber	Nutzer wählt Bild/Video aus der Galerie. Nutzt Androids fertigen Auswahl-Dialog → braucht keine Speicherberechtigung.
CameraX (eigener Aufnahme-Screen)	Foto-/Videoaufnahme beim Anlegen einer Route und im Ghost Climber — KameraScreen, CameraCaptureController (einzige Stelle mit CameraX), Quellenwahl über MedienQuelleDialog.	Statt den Kamera-Intent aufzurufen (Ergebnis je nach Gerät unterschiedlich), nimmt die App selbst auf — mit fest gewählter Auflösung (Quality.HD). Das macht besonders die Ghost-Climber-Pipeline berechenbarer und legt die Aufnahmen als app-eigene FileProvider-URI ab, damit der Pose-Cache stabil bleibt. Braucht die CAMERA-Berechtigung.
Media3 / ExoPlayer	Video-Abspieler im Boulder-Detail und im Ghost Climber (Videoplayer)	Spielt die zu einer Route gespeicherten Videos ab. Stoppt automatisch, wenn man den Screen verlässt (kein Weiterlaufen im Hintergrund).
Sensoren (Beschleunigung)	Auf der Smartwatch, während des Hangboard-Trainings (AutoHangService)	Erkennt automatisch aus der Armbewegung, wann man hängt und wann Pause ist — ohne dass man selbst tippen muss.
Foreground Service	Die Sensor-Erfassung auf der Uhr läuft als Vordergrund-Dienst	Damit die Messung weiterläuft, auch wenn das Uhr-Display ausgeht (Android beendet normale Hintergrund-Arbeit

Feature	Where it's used (screen/class)	Why this feature was needed
		sonst).
Wear Data Layer	Verbindung Uhr ↔ Handy (PhoneConnector auf der Uhr, HangboardWearListenerService am Handy)	Die Uhr schickt fertige Hangboard-Durchläufe ans Handy; das Handy schickt gespeicherte Timer-Vorlagen zurück.
Homescreen Widget (Glance)	Widget auf dem Android-Startbildschirm (BoulderWidget)	Zeigt die laufende Session direkt am Homescreen und springt per Tipp in die App.
Adaptive Layouts	Automatische Umschaltung Handy ↔ Tablet (MainActivity, WindowSizeClass)	Auf dem Tablet stehen Session-Liste und Detail nebeneinander, auf dem Handy untereinander. Erfüllt die „Responsive“-Anforderung.
Type-safe Navigation	Bildschirm-Wechsel in der ganzen App (Destinations)	Regelt, wie man zwischen den Screens navigiert und Daten (z. B. welche Session) sicher mitgibt.
System-Spracheingabe	Mikrofon-Knopf in den Notizfeldern (Session anlegen, Route hinzufügen) — SpeechToTextButton mit eigenem SpeechInputDialog (Live-Teiltext + Pegelring); SystemSpeechRecognitionClient als Wrapper um die SpeechRecognizer-API.	Notiz diktieren statt tippen — mit eigener Aufnahme-Oberfläche statt des fremden Google-Dialogs. Läuft über eine zweistufige Kette: On-Device-Erkennung (offline, die Notiz verlässt das Gerät nicht), sonst Netz-Erkennung. Ist kein Erkennungsmodell verfügbar, erscheint statt eines stillen System-Dialogs eine ehrliche Meldung mit „Modell laden“-Knopf — der frühere System-Dialog-Fallback wurde bewusst entfernt, weil er die Mikrofon-Ablehnung des Nutzers umgangen hätte. Dafür braucht die App erstmals eine Laufzeit-Berechtigung (RECORD_AUDIO).
MediaStore-Export	„Session exportieren“ (SessionExporter)	Schreibt die Session-Daten als CSV in den Download-Ordner, damit man sie außerhalb der App weiterverwenden kann.
Standort & Geofencing (FusedLocationProviderClient + Geofencing)	Gym-Näherungs-Push (proximity-Paket: GeofenceManager, Receiver, GymBearbeiten mit „Standort übernehmen“)	Erkennt energiesparend, wenn man sich länger an einer gespeicherten Kletterhalle aufhält, und stößt die Erinnerung an — ohne die App offen zu halten.
Benachrichtigungen	„Bist du in der Halle? Session	Zeigt die Erinnerung mit direktem

Feature	Where it's used (screen/class)	Why this feature was needed
(Notification-Manager)	starten?"-Push	Sprung in den vorbefüllten Session-Screen (Deep-Link).
Nearby Connections (P2P)	Geräte-Abgleich Handy ↔ Tablet (sync/nearby, AbgleichViewModel)	Gleicht den kompletten Datenstand direkt zwischen zwei Geräten ab — ohne Konto oder Server, beide Stände liegen gleichzeitig vor.

5.2. Implementation Details

1. Automatische Hangboard-Erkennung auf der Smartwatch

was und warum: Beim Hangboard-Training (hängen an einer Trainingsleiste) soll die Uhr selbst erkennen, wann der Nutzer hängt und wann er pausiert — ohne manuelles Tippen. Grundlage sind die Bewegungsdaten des Handgelenks.

Umsetzung: Ein `AutoHangService` (ein Android-Service) holt sich über den `SensorManager` zwei virtuelle Sensoren: `TYPE_GRAVITY` (die Richtung, in die der Arm zeigt) und `TYPE_LINEAR_ACCELERATION` (die reine Bewegung ohne Erdbeschleunigung), abgetastet mit `SENSOR_DELAY_GAME` (~50 Hz). Die eigentliche Erkennungslogik (`HangDetection`) ist bewusst frei von Android-Abhängigkeiten gehalten — sie bekommt nur Zahlenwerte herein und ist dadurch mit normalen JVM-Unit-Tests prüfbar, ohne Gerät.

Lebenszyklus: Weil eine solche Messung weiterlaufen muss, obwohl das Uhr-Display längst aus ist, läuft der Dienst als `Foreground Service` (ein vom System sichtbar gehaltener, nicht beliebig beendbarer Hintergrunddienst) vom Typ `health`. Ein kurzer `PARTIAL_WAKE_LOCK` verhindert, dass die CPU beim ruhigen Hängen einschläft. Beim Beenden gibt `onDestroy()` beides wieder frei (`unregisterListener`, `Wakelock-Release`), damit nach dem Training kein Akku mehr verbraucht wird.

Berechtigungen: Der `health-Foreground-Service` verlangt eine `Sensor-Berechtigung`; gewählt wurde `HIGH_SAMPLING_RATE_SENSORS` — eine `Install-Time-Permission` (kein Laufzeitdialog) statt des dialogpflichtigen `BODY_SENSORS`. Dazu `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_HEALTH` und `WAKE_LOCK` im Manifest.

Grenzfälle: Fehlt ein Sensor (z. B. im Emulator, der die virtuellen Sensoren `GRAVITY/LINEAR_ACCELERATION` nicht bereitstellt), bricht der Dienst nicht ab, sondern protokolliert eine Warnung (`Log.w`) und läuft weiter — sichtbar würde das nur als „erkennt nichts“.

2. Übertragung Uhr → Handy und dauerhaftes Speichern (Wear Data Layer + Room)

Was & warum: Ein auf der Uhr abgeschlossenes Hangboard-Workout soll auf dem Handy in der Trainingshistorie landen — möglichst der gerade laufenden Kletter-Session zugeordnet.

Umsetzung: Die Uhr sendet das Ergebnis über den `Wear Data Layer` — Androids Standard-Kanal zwischen gekoppelten Geräten — per `MessageClient` (`PhoneConnector` auf der Uhr). Auf dem Handy empfängt ein `HangboardWearListenerService` (ein `WearableListenerService`) die Nachricht und schreibt sie in die Datenbank. Umgekehrt schickt das Handy gespeicherte Timer-Vorlagen über einen `DataClient` (dauerhaft gecachtes Datenelement) zurück an die Uhr.

Speicherlogik („immer speichern, verknüpfen wenn möglich“): Der Service fragt die aktive Session ab (`sessionRepository.observeActive()`). Läuft gerade eine, wird das Workout an sie gehängt; läuft keine, wird es als eigenständiges Training gespeichert (`sessionId = null`) statt verworfen. In der Datenbank bildet das ein vereintes Modell ab: `HangboardWorkoutEntity` (mit mode `MANUAL/AUTO` und origin `PHONE/WATCH`) plus zugehörige `HangboardSegmentEntity`-Zeilen (die einzelnen Sätze), zusammengeführt über eine `@Relation`.

Berechtigungen: Der Empfänger-Service nutzt Dependency Injection (`@AndroidEntryPoint/Hilt`); der dadurch generierte Service-Code benötigte unter AGP 9 zusätzlich die Bibliothek `error_prone_annotations`, sonst schlägt der Build fehl (siehe 5.3).

Grenzfälle: Ist kein Handy gekoppelt, funktioniert die Uhr weiter als reiner Timer (Companion-Prinzip, `standalone = false`) — nur die Übertragung entfällt. Der Datenkanal-„Vertrag“ (Pfad + Nachrichtenformat) existiert bewusst doppelt auf beiden Geräten und muss manuell synchron gehalten werden.

3. Lebenszyklus-bewusste Video-Wiedergabe (Media3 / ExoPlayer)

Was & warum: Zu jeder Route lässt sich ein Video speichern und im Detail-Screen abspielen. Ein Video-Player belegt Systemressourcen (Decoder, Speicher), die sauber freigegeben werden müssen — sonst spielt Ton im Hintergrund weiter oder es entstehen Speicherlecks.

Umsetzung: `VideoPlayer` kapselt einen `ExoPlayer` (aus `Media3`) und bindet dessen View-basierte `PlayerView` über `AndroidView` in die ansonsten in Jetpack Compose geschriebene Oberfläche ein.

Lebenszyklus: Ein `DisposableEffect` mit einem `LifecycleEventObserver` koppelt den Player an den Bildschirm-Lebenszyklus: Beim Ereignis `ON_STOP` (App wird verlassen / Screen nicht mehr sichtbar) wird `pause()` aufgerufen, beim endgültigen Verlassen im `onDispose` wird der Player mit `release()` vollständig abgebaut. So gibt es kein Weiterspielen im Hintergrund und keine liegengebliebenen Decoder.

Grenzfälle: Ob eine Datei ein Bild oder ein Video ist, wird nicht in der Datenbank gespeichert, sondern zur Laufzeit über `contentResolver.getType(uri)` bestimmt — so kann kein Feld von der tatsächlichen Datei abweichen.

5.3. AI's Role in Android-Specific Code

Wo die KI korrekt und hilfreich war:

1. Ein schwer auffindbarer Build-Fehler (Hilt + AGP 9). Sobald der `HangboardWearListenerService` (ein per Dependency Injection versorgter `WearableListenerService`) hinzukam, brach der Build mit einer irreführenden Meldung über ein fehlendes Paket `com.google.errorprone.annotations`. Die KI führte das korrekt auf den generierten Service-Code zurück (nur der `ServiceComponent` referenziert `@CanIgnoreReturnValue`, nicht die üblichen `ViewModel`-/Activity-Komponenten) und ergänzte gezielt die Abhängigkeit `error_prone_annotations` — heute als fester Katalogeintrag im Build.
2. Korrektes Berechtigungsmodell für Hintergrunddienste. Für die Sensor-Erfassung auf der Uhr schlug die KI einen `Foreground Service` vom Typ `health` in Kombination mit der Install-Time-Berechtigung `HIGH_SAMPLING_RATE_SENSORS` vor — statt der dialogpflichtigen

BODY_SENSORS. Das ist die passende, reibungsärmere Wahl und im Manifest genau so umgesetzt.

3. Sauberer Umgang mit dem Wear Data Layer. Die Aufteilung in MessageClient (einmalige Nachricht Uhr→Handy) und DataClient (dauerhaft gecachtes Datenelement für die Timer-Vorlagen Handy→Uhr) entspricht der von Google empfohlenen Nutzung und wurde ohne Nacharbeit übernommen.

Wo nachgebessert werden musste (falsche Annahmen)

Die Android-spezifischen Fehlannahmen betrafen weniger „veraltete APIs“ als falsche Lebenszyklus-Annahmen rund um die Pose-Bibliothek des Ghost Climbers:

1. Verwechslung von „im Bild“ mit „Position korrekt“. Der erste Ansatz nutzte ML Kits inFrameLikelihood als Qualitätsmaß. Dieser Wert sagt aber nur aus, dass ein Gelenkpunkt im Bild liegt, nicht dass seine Position stimmt — verdeckte Gliedmaßen bekamen an falscher Stelle hohe Werte. Korrigiert durch den Wechsel auf MediaPipe, das echte visibility/presence-Werte liefert.
2. Übersehener interner Zustand von RunningMode.VIDEO. MediaPipe im Video-Modus führt ein internes Tracking über die Frames hinweg. Die anfängliche Annahme, ein einzelnes „Fremdbild“ (ein Vollbild zwischen den Ausschnitten) störe nur diesen einen Frame, war falsch: Es zog auch den Folgeframe mit — sichtbar als Zittern „einmal pro Sekunde“. Das ließ sich erst durch Auswertung der Rohdaten vom Gerät belegen und wurde dann behoben (Diagnose-Erkennung in einer separaten RunningMode.IMAGE-Instanz).
3. Stolpersteine bei der Bibliotheks-Anbindung. Mehrere MediaPipe-Eigenheiten mussten recherchiert und im Code abgesichert werden: Es akzeptiert nur ARGB_8888-Bitmaps (nicht das je nach Gerät gelieferte RGB_565), verlangt streng aufsteigende Zeitstempel, und lädt sein Modell — anders als ML Kit — nicht selbst nach, sondern erwartet es als mitgeliefertes Asset im App-Paket.

6. Lessons Learned

- Technical insights:

Die Plattform macht mehr Arbeit als die eigene Idee. Lebenszyklus, Berechtigungen und versionsabhängige APIs sind die eigentlichen Zeitfresser – am besten zentral im Code kapseln, statt sie an vielen Stellen einzeln zu lösen.

Nicht optimieren, was man nicht misst. Ohne eine konkrete Kennzahl sollte man einen Teil nicht optimieren, nur weil ein anderer Teil des Features optimiert werden musste. Das kann dazu führen, dass bereits funktionierende Teile zu Fehlern „optimiert“ werden.

- Architectural insights:

Einzelne UI-Elemente (Button, BottomNavBar, TopBar usw.) in eigenen Dateien bauen. Die Elemente sind dadurch alle fertig vorhanden und dienen als Baukasten, um daraus die UI-Screens zusammenzusetzen. Das macht den Code deutlich übersichtlicher:

Jedes Element wird nur einmal geschrieben, lässt sich beliebig oft wiederverwenden, und ein einheitliches Design ist garantiert.

- UX insights:

Detaillierte UI-Ideen in konkreten Bildern festhalten, um die UI-Erstellung im Code zu erleichtern. Eine konkrete, detaillierte Ausarbeitung – nicht nur grobe Wireframes – zeigt genau, welche UI-Elemente benötigt werden und wie diese am Ende aussehen sollen.

- AI-related insights:

Auch eine KI kann schnell den Überblick im Projekt verlieren und baut dadurch Fehler ein. Hat sie nur das Projekt als Ganzes vor sich und keine konkrete Struktur, an der sie sich entlanghangeln kann, macht sie schnell zu viel auf einmal – und dabei schleichen sich kleinere oder auch größere Fehler ein. Genau das hat uns zur nächsten Lesson Learned geführt.

Konkrete Schritt-für-Schritt-Anleitungen erstellen. So bekommt die KI ihre Arbeit in vielen kleinen Teilaufgaben zerlegt, die sie einzeln – und dadurch mit deutlich weniger Fehlern – abarbeiten kann.

- What would you do again or differently?

Einen Obsidian-Vault strukturiert und gepflegt nutzen. Claude kann direkt in einem Obsidian-Vault arbeiten und darin schreiben. So lassen sich viele Arbeitsschritte – wie z. B. das Dokumentieren der KI-Nutzung – automatisiert festhalten. Zudem liefert der Vault eine gute Übersicht und kann durch seine Verlinkungs-Funktion zu einer sehr verständlichen Dokumentation führen.

KI-Ausgaben immer überprüfen und korrigieren. Das haben wir von Anfang an so gehandhabt und würden es wieder so machen: Eine KI liefert plausible, aber nicht zwangsläufig korrekte Ergebnisse – erst das eigene Prüfen und Nachbessern macht sie verlässlich.

7. Future Work

- Possible refactoring:
- Possible extensions: